김효식 포트폴리오

백엔드 엔지니어

문제의 원인을 파악하고,
구조적 개선을 통해 안정적인 흐름을 만드는 개발자입니다.
직접 검증하며 성능을 비교하고, 더 나은 해결 방법을 끝까지 고민합니다.

문제 정의 > 구조 개선 > 검증

Mail. gytlr339@gmail.com | Tel. 010-4916-2786 | Github. Link | Blog.Link

프로젝트

쇼핑몰 API 서버

1인 개발 · 2026.01 ~ 2026.02 · 백엔드 설계 및 구현

개요

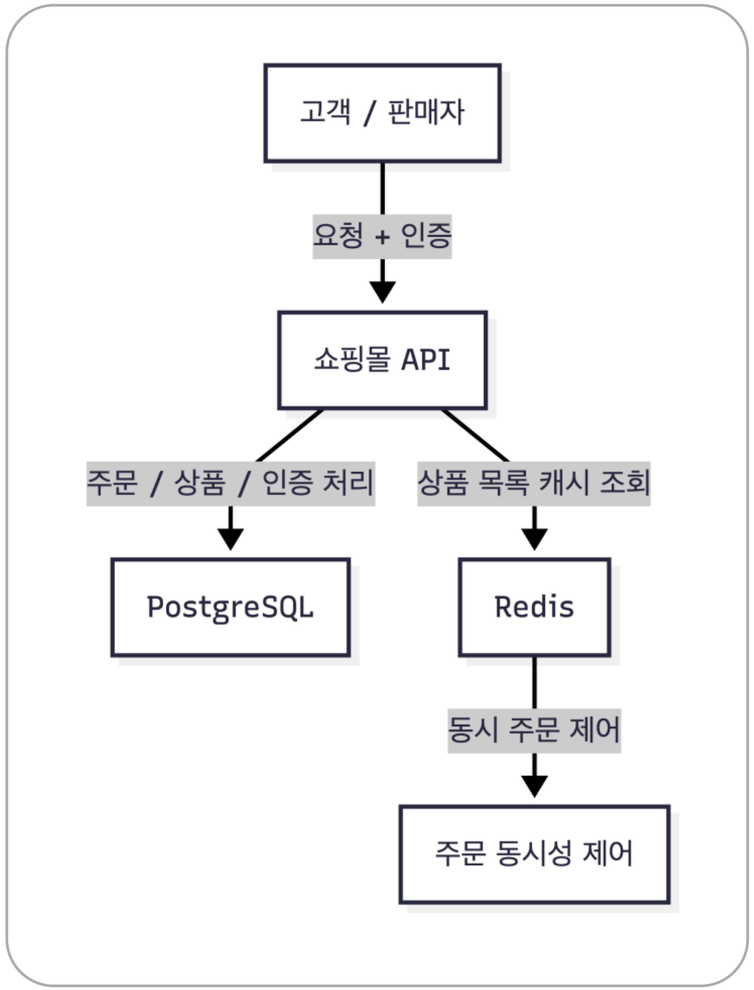
주문 정합성과 조회 성능을 함께 개선한 쇼핑몰 서버 프로젝트

주요 기능

- 주문 동시성 제어를 통한 안정적인 재고 처리
- 상품 목록 캐시 적용을 통한 빠른 조회 응답
- 성능 테스트 및 지표 분석을 통한 기술 스택 비교 검증

기술

Spring Boot/
Redis / PostgreSQL



▲ 서버 아키텍처 다이어그램

문제

동시 주문 환경에서 재고 정합성 보장

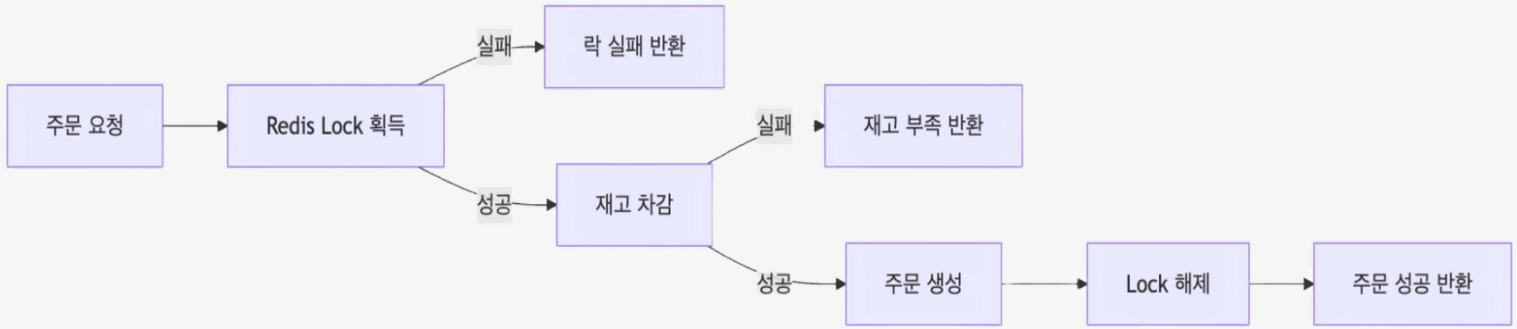
문제 상황

동시 주문 시 동일 재고가 **중복 차감**되어 **실제 재고와 시스템 재고간의 불일치** 가능

접근 방식

상품 단위 락으로 **동시성**을 제어하고, 재고 **중복 차감**을 방지,
실패 원인을 **락 획득 실패**와 **재고 부족**으로 구분해 재시도 흐름까지 안정화

처리 흐름



검증 결과

동시성 테스트에서도 음수 재고 없이 **최종 재고 정합성**을 유지,
실패 흐름 역시 **락 실패**와 **재고 부족**으로 명확히 분리

배운점/교훈

동시성 제어는 단순히 요청을 막는 문제가 아니라,
실패를 예측 가능하게 다루도록 **설계하는하는 문제**라는 점을 배웠습니다.

성능 비교

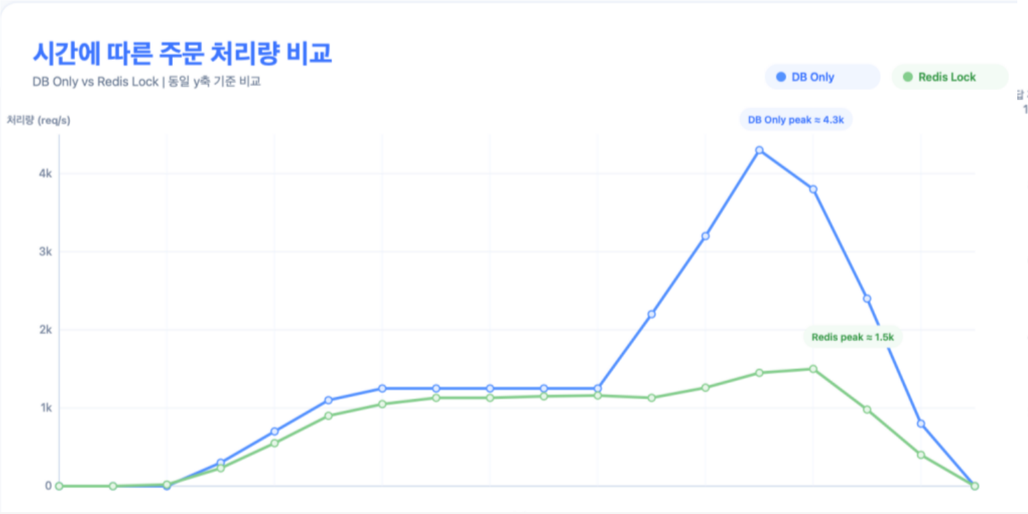
성능 비교를 통한 동시성 제어 방식 선택

검증 방식

동일 상품에 주문이 집중되는 경합 시나리오를 구성한 뒤, **JMeter**와 **Grafana**로 재고 차감 전략별 처리 성능을 비교 검증

1. Redis 락 + DB 조건부 차감
2. DB 조건부 차감만 적용

검증 결과



현재 단일 인스턴스 구조에서는 DB 조건부 차감만 적용한 방식이 더 효율적이었습니다.
[Grafana 관측 기준] **최대 처리량 :** 약 1.5k req/s → 4.3k req/s (약 2.9배)
[Grafana 관측 기준] **응답 지연** : p95 200~450ms → 150~220ms, p99 370~820ms → 220~315ms

배운점/교훈

동시성 제어 방식은 복잡한 기술을 우선 적용하기보다,
현재 구조에서 가장 **단순**하면서도 **성능과 정합성**을 함께 만족하는 방식을 검증 후 선택해야 한다는 점을 배웠습니다.

문제

캐싱을 통한 반복 조회 성능 최적화

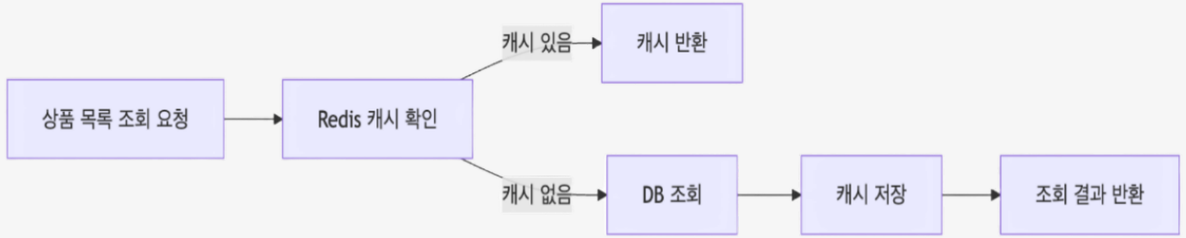
문제 상황

반복되는 목록 조회가 모두 DB로 향해 읽기 부하와 응답 지연이 발생 가능

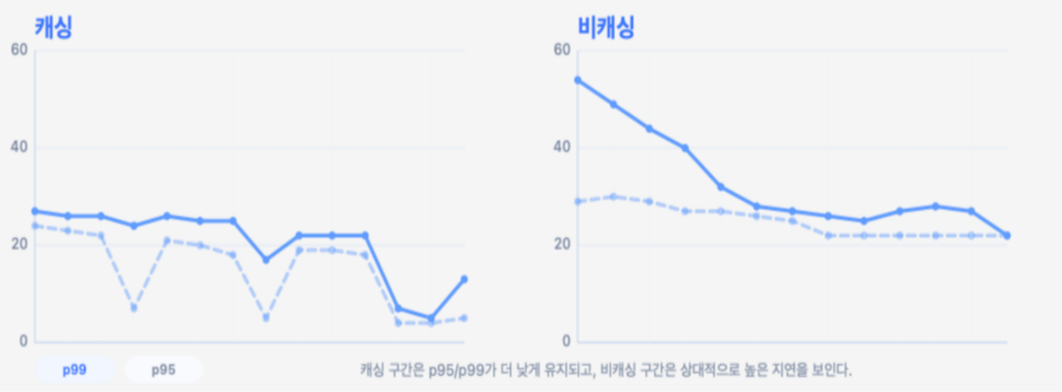
접근 방식

검색 조건 기반 Redis 캐시와 TTL, 변경 시 캐시 무효화, Redis 장애 시 DB fallback을 적용

처리 흐름



검증 결과



캐시 적용 시 p95/p99가 더 낮고 안정적으로 유지

배운점/교훈

캐싱은 응답 속도를 개선 기능이 뿐만 아니라, 어떤 조건을 키로 삼고 언제 무효화할지까지 포함한 **데이터 일관성 설계**라는 점을 배웠습니다.

프로젝트

티켓팅 서버

1인 개발 · 2026.02 · 백엔드 설계 및 구현

개요

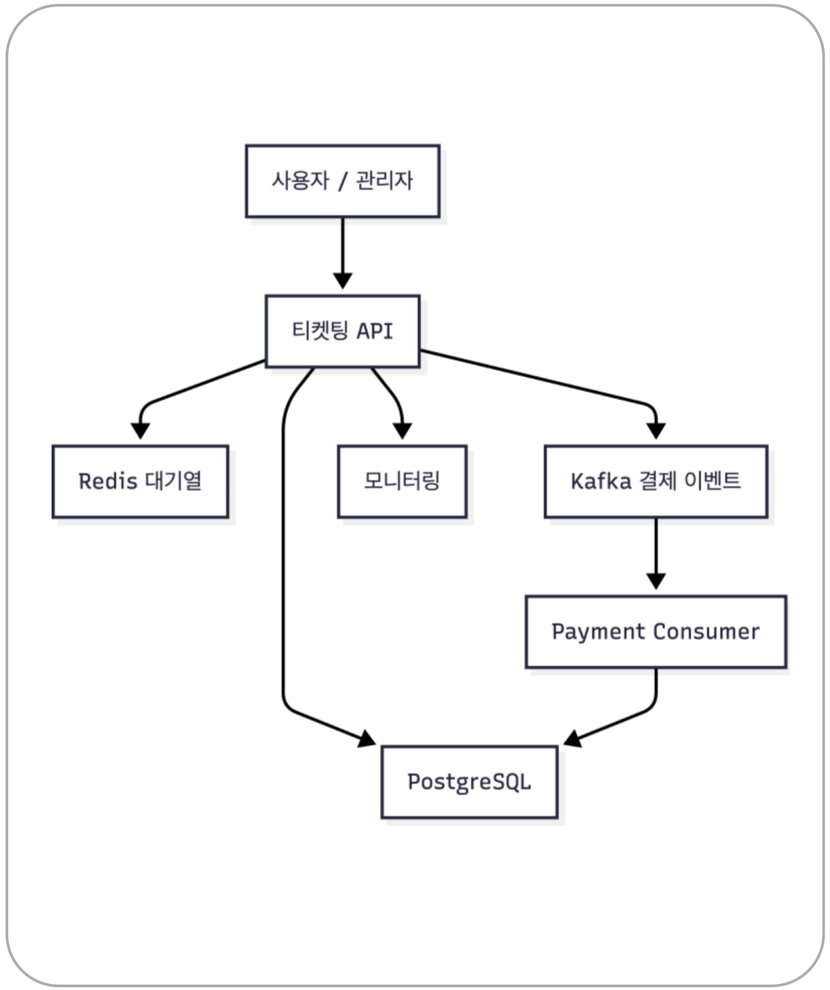
대기열과 비동기 결제로 혼잡 상황의 예매 흐름을 제어한 서버 프로젝트

주요 기능

- 대기열 기반 동시 접속 제어를 통한 안정적인 예매 처리
- 비동기 결제 요청 처리와 트랜잭션 기반 재고 차감으로 정합성 확보
- 중복 요청 방지 및 운영 모니터링을 통한 예매 시스템 안정성 강화

기술

Spring Boot/ Kafka
Redis / PostgreSQL



▲ 서버 아키텍처 다이어그램

문제

대규모 동시 접속 티켓팅 대응

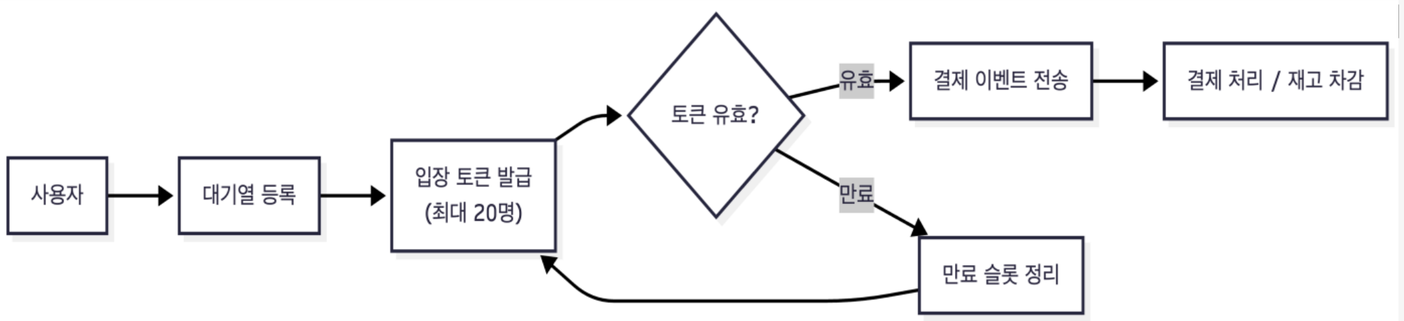
문제 상황

예매 오픈 시 다수 사용자가 동시에 결제 구간에 몰리며 서버 병목, 중복 요청, 재고 정합성의 문제 발생 가능

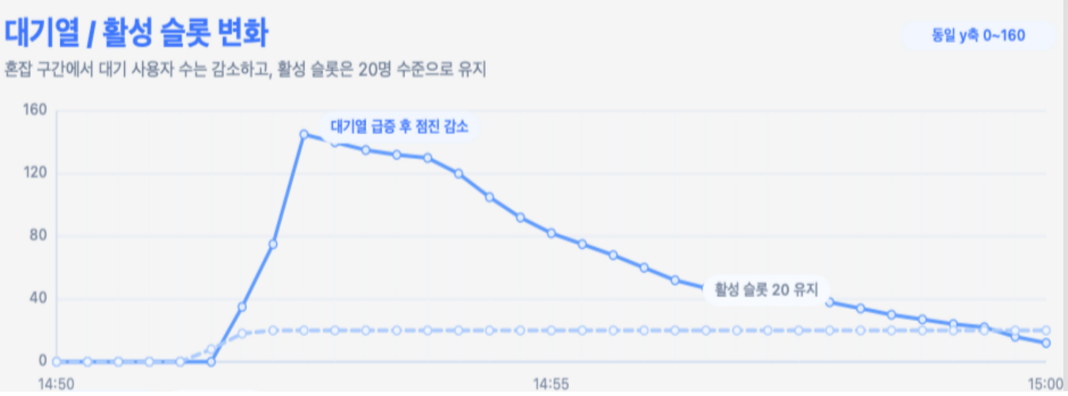
접근 방식

Redis 대기열로 사용자 순번을 관리하고 입장 토큰으로 결제 가능 인원을 제한
Lua 스크립트로 토큰 발급을 원자적으로 처리
Kafka 비동기

처리 흐름



검증 결과



JMeter 200명 혼합 시나리오 기준

혼잡 구간에서 대기열은 일시적으로 약 140명까지 증가, 활성 슬롯은 20명 수준으로 안정적으로 유지
이후 대기열이 지속적으로 감소해 종료 시점에는 거의 해소되며, 동시 진입 제어와 처리 안정성을 확인

배운점/교훈

캐싱은 응답 속도를 개선 기능이 뿐만 아니라, 어떤 조건을 키로 삼고 언제 무효화할지까지 포함한 **데이터 일관성 설계**라는 점을 배웠습니다.

관측

대기열 기반 티켓팅 제어 : 대시보드

시나리오 : 대기열의 70% 결재(슬롯 반환), 30% (토큰 만료)

지표 1

활성 슬롯이 20명 수준으로 유지되는가?

지표 2

대기 사용자 수만 감소하고 결제 이벤트 발행량은 급증하지 않는지 확인

지표 3

슬롯 반환과 만료가 정상적으로 순환하는지 확인